PROBLEMA

La raíz que se esconde

Comparación de métodos de búsqueda de raíces y su convergencia

Universidad Católica del Maule - Laboratorio evaluado 2

Unidad: Iteración funcional para ecuaciones — búsqueda de raíces y análisis de convergencia.
Métodos: bisección, falsa posición, punto fijo, secante, Newton-Raphson y Newton-Raphson mejorado.
Tolerancia: `1e-6` como criterio de parada, con tope de iteraciones.
Modalidad: en parejas (máximo 2 personas) o individual. Entrega por repositorio GitHub.

1. Objetivo

Trabajando en parejas (máximo 2) o de forma individual, deberán implementar y comparar seis métodos de búsqueda de raíces sobre funciones de distinta naturaleza y mostrar, con datos y gráficos, en qué se diferencian al converger. En concreto, deberán:

- Implementar los seis métodos y aplicarlos sobre funciones polinómicas, logarítmicas, exponenciales, trigonométricas y una mixta.
- Demostrar que cada función tiene raíz dentro del rango (cambio de signo) o que toca el eje x (raíz doble), dentro de un ambiente controlado.
- Registrar la convergencia de cada método (iteraciones, raíz y error) con tolerancia `1e-6`.
- Responder, con sus propios datos y gráficos, las preguntas de convergencia de la sección 6.

2. Contexto — por qué comparar métodos —

Encontrar la raíz de una ecuación es responder «¿para qué valor de x la función vale cero?». Casi nunca hay una fórmula directa, así que se usan métodos que se acercan al cero paso a paso. El detalle es que no todos cuestan lo mismo: algunos son lentos pero seguros (bisección), otros son rapidísimos pero pueden fallar o irse a otra raíz (Newton, secante), y otros dependen de cómo se plantee la iteración (punto fijo).

La gracia de esta evaluación no es solo «encontrar el cero», sino medir cuánto cuesta encontrarlo con cada método —cuántas iteraciones, con qué error— y entender por qué unos ganan y otros se atascan. El caso más interesante es cuando la curva toca el eje sin cruzarlo (una raíz doble): ahí varios métodos simplemente no sirven.

3. Las funciones a probar

Deben probar al menos una función de cada categoría (mínimo 5), e incluir una con raíz doble que toque el eje x. Pueden usar estas o proponer las suyas:

Categoría	Función de ejemplo	Intervalo [a, b] / x_0
Polinómica (raíz simple)	$f(x) = x^3 - x - 2$	[1, 2]
Polinómica (raíz doble)	$f(x) = (x - 2)^2$	$x_0 = 1{,}2$ (no hay cambio de signo)
Logarítmica	$f(x) = \ln(x) + x - 4$	[2, 3]
Exponencial	$f(x) = e^{(-x)} - x$	[0, 1]
Trigonométrica	$f(x) = \cos(x) - x$	[0, 1]

Categoría	Función de ejemplo	Intervalo [a, b] / x_0
Mixta	f(x) = √x − cos(x)	[0.1, 1]

4. El ambiente controlado

Para que los resultados sean reproducibles y demostrables, cada función se entrega con un ambiente controlado:

- Tamaño de paso: declarar el intervalo [a, b] (métodos cerrados) o la estimación inicial x_0 y x_1 (secante).
- Fijar la tolerancia en `1e-6` y un tope de iteraciones (por ejemplo 100).
- Demostrar que la raíz está en el rango: cambio de signo f(a)·f(b) < 0 (Bolzano) para las que cruzan; tangencia al eje para la raíz doble.
- Graficar cada función con la raíz marcada.

5. Definiciones y criterios

- **Criterio de parada:** $|f(x_n)| < tol$ o $|x_n - x_{n-1}| < tol$.
- **Error:** $|x_n - raíz|$ (si se conoce la raíz) o $|x_n - x_{n-1}|$.
- **Orden de convergencia:** lineal (bisección, falsa posición, punto fijo), superlineal (secante ≈ 1,618) y cuadrático (Newton y Newton mejorado en raíces simples).
- **Cerrados vs abiertos:** los métodos cerrados (bisección, falsa posición) exigen cambio de signo; los abiertos (punto fijo, secante, Newton) parten de estimaciones y pueden fallar.

6. Preguntas de convergencia (en informe.md)

El informe debe responder, con sus propios datos y gráficos:

- ¿Qué método convergió en menos iteraciones en cada función? ¿Por qué?
- ¿Qué orden de convergencia tiene cada método y cómo se refleja en el gráfico |error| vs iteración?
- ¿Qué métodos requieren un intervalo con cambio de signo y cuáles no?
- En la función que toca el eje x (raíz doble): ¿qué métodos fallan o no aplican y cuál se comporta mejor? ¿Por qué?
- ¿Cómo influyen el intervalo inicial y la estimación inicial en la cantidad de iteraciones y en si el método converge?
- ¿Qué método recomiendan según el tipo de función, considerando el compromiso entre velocidad y robustez?

7. Entregable: repositorio GitHub

Repositorio (público o compartido con el docente) con esta estructura:

```
evaluacion2-busqueda-raices/
├── README.md          <- entorno (venv o Docker) y qué hace cada script
├── requirements.txt   <- numpy, matplotlib
├── informe.md         <- respuestas a las preguntas de convergencia
└── src/
    ├── assets/        <- gráficos PNG generados
```

```
    ├── data/          <- CSV por función con columnas (x, f(x))
    └── scripts/       <- todos los scripts de Python
```

Cálculo con numpy (obligatorio): implementar los métodos y las funciones con numpy, y generar los CSV de datos y el resumen de convergencia.

Gráficas obligatorias (matplotlib):

- Cada función con su raíz marcada (y, en la raíz doble, la tangencia al eje).
- Convergencia |error| vs iteración (escala log) comparando los métodos por función.
- Un resumen de iteraciones por método y función.

8. Reglas de ramas en GitHub

El repositorio debe tener 2 o 3 ramas: una `main` con el código evaluado y una rama por colaborador. **El código evaluado es únicamente el que esté en main**: si queda en la rama de un colaborador y no se integra a main, no se considerará.

```
git checkout -b colaborador1     # cada integrante trabaja en su rama
git checkout main
git merge colaborador1           # integrar a main lo que se evalúa
git push origin main
```

9. Documento de entrega (informe.md) y conclusión

El informe.md debe cerrar con una conclusión argumentada que responda:

- ¿Qué método elegirían para cada tipo de función y por qué (velocidad vs robustez)?
- ¿Qué ocurrió en la raíz doble y qué dice eso sobre «tener raíz» frente a «cambiar de signo»?
- ¿Cómo cambió el número de iteraciones al mover el intervalo o la estimación inicial?

La conclusión debe apoyarse en los números y gráficos obtenidos, no en la intuición.

10. Rúbrica de evaluación

10 criterios (8 técnicos + 2 de habilidades blandas). Puntaje por criterio: **5 · 3 · 1 · 0**. Puntaje máximo: 50 puntos. Marque en cada fila el nivel alcanzado.

Criterio	5 puntos — Logrado	3 puntos — Parcial	1 punto — Incipiente	0 puntos — No logrado
T1. Implementación de los métodos *(Técnico)*	Los seis métodos (bisección, falsa posición, punto fijo, secante, Newton-Raphson y Newton mejorado) están implementados, son correctos y entregan la raíz esperada donde aplican.	Cuatro o cinco métodos funcionan bien; uno o dos tienen errores menores o no convergen en algún caso.	Solo dos o tres métodos funcionan; el resto tiene errores de implementación.	Ninguno o un solo método funciona, o el código no se ejecuta.
T2. Variedad de funciones *(Técnico)*	Incluye al menos una función de cada categoría (polinómica, logarítmica, exponencial, trigonométrica y mixta) y además una con raíz doble que toca el eje x.	Incluye las cinco categorías, pero sin el caso de raíz doble.	Faltan dos o más categorías de funciones.	Usa una sola función o no cumple el mínimo.
T3. Ambiente controlado *(Técnico)*	Declara intervalo o estimaciones iniciales y tolerancia 1e-6 en todas las funciones, y demuestra la raíz (cambio de signo o tangencia al eje) con respaldo gráfico.	El ambiente está definido pero incompleto en algunas funciones (falta un intervalo, una demostración o la tolerancia explícita).	Solo menciona el ambiente, sin demostrar que la raíz está en el rango.	No define un ambiente controlado.
T4. Datos generados (CSV) *(Técnico)*	Entrega un CSV por función con columnas (x, f(x)) y un CSV resumen de convergencia (iteraciones, raíz y error) correctos.	Entrega los CSV pero con omisiones (falta alguna función o el resumen de convergencia).	Solo un CSV parcial o con datos inconsistentes.	No entrega datos en CSV.
T5. Gráficos *(Técnico)*	Grafica cada función con su raíz marcada y la convergencia |error| vs iteración comparando métodos; todo rotulado (ejes, leyenda y título).	Presenta ambos tipos de gráfico pero con deficiencias (sin leyenda, ejes sin rótulo o baja legibilidad).	Solo un tipo de gráfico, o gráficos ilegibles.	No incluye gráficos.
T6. Análisis de convergencia *(Técnico)*	Responde las seis preguntas de convergencia con argumentos correctos y respaldo en sus propios datos y gráficos.	Responde la mayoría de las preguntas, con fundamento parcial.	Respuestas superficiales o sin respaldo numérico ni gráfico.	No presenta análisis de convergencia.
T7. Estructura, README y entorno *(Técnico)*	Estructura de carpetas correcta; el README explica el entorno (venv o Docker) y qué hace cada script;	Funciona, pero con pasos faltantes o confusos en el README o el entorno.	Estructura o README muy incompletos; difícil de reproducir.	No se puede montar ni ejecutar el proyecto.

Criterio	5 puntos — Logrado	3 puntos — Parcial	1 punto — Incipiente	0 puntos — No logrado
	requirements.txt correcto y el proyecto se reproduce siguiendo el README.			
T8. Git/GitHub y ramas *(Técnico)*	Repositorio con main más una rama por colaborador; el código evaluado está integrado en main y el historial de commits es coherente.	Crea las ramas, pero la integración a main es incompleta.	Una sola rama, o el código evaluado quedó fuera de main.	Sin control de versiones adecuado o sin repositorio.
B1. Trabajo colaborativo *(Blanda)*	El aporte de cada integrante es claro y equilibrado, visible en los commits de cada uno (en individual, avance sostenido y organizado).	Ambos integrantes aportan, pero de forma desigual.	Casi todo el trabajo recae en una sola persona.	No hay evidencia de colaboración ni organización.
B2. Comunicación y claridad *(Blanda)*	Informe y README claros, ordenados y bien redactados, con ortografía cuidada y lenguaje técnico correcto.	Comprensible, con descuidos menores de redacción u ortografía.	Desordenado o difícil de seguir.	No comunica el trabajo realizado.

Conversión sugerida a nota (1,0–7,0): nota = 1,0 + 6,0 × (puntaje / 50), o según la tabla de exigencia 60 % del CFT.